Frontend Developer Assignment

Objective

Build a polished, production-minded frontend application that demonstrates your ability to create polished user interfaces, render complex markdown accurately, and create an intuitive user experience. The assignment should be completed within approximately 4-6 hours. We do not expect every possible enhancement to be implemented; prioritize a well-finished core experience.

This assignment is designed to evaluate your frontend engineering skills, UI/UX design decisions, code quality, responsiveness, and attention to detail.

There are no design mockups provided. You are expected to make your own design decisions and build a clean, professional application.

Submit the deliverables outlined below via this Google Form: https://forms.gle/TbcK8kXayS7UCU7h8.

Assignment

Within 4-6 hours, build a markdown rendering application capable of rendering complex markdown documents generated by AI models. Use GitHub-Flavored Markdown (GFM) as the baseline specification.

Required Markdown support:

- Headings
- Paragraphs
- Ordered lists
- Unordered lists
- Nested lists
- Tables
- Blockquotes
- Inline code
- Code blocks
- Bold
- Italic
- Strikethrough
- Hyperlinks

Your application should:

- Allow users to upload one markdown file at a time.
- Render the markdown accurately and consistently.
- Gracefully handle malformed or incomplete markdown without crashing.
- Produce a clean, readable, and polished output.
- Implement a single Copy button that writes both HTML and plain-text representations of the rendered content to the clipboard. The destination application should automatically use the richest format it supports.
    - Also include the original Markdown as a text/markdown clipboard representation where browser support permits.
    - The copied HTML should preserve useful formatting when pasted into applications such as Microsoft Word, Google Docs, or rich-text editors, with a plain-text fallback where appropriate.

Sample markdown files will be provided for reference. The final evaluation will be performed using hidden test files. Hidden tests will focus on the required features and reasonable edge cases.

Technical Requirements

- React.js is mandatory.
- JavaScript only. TypeScript is not allowed.
- Tailwind CSS is mandatory for styling.
- You may use any React-based framework (e.g., Next.js or Vite).
- You may use any open-source libraries where appropriate.
- No backend is required.
- Everything should run entirely in the browser.

Final Deliverables

- Public GitHub repository containing the complete source code.
- **Publicly hosted application (e.g., Vercel, Netlify, Cloudflare Pages, etc.) and provide a link to access the application.**
- A concise text that explains setup instructions, key design and technical decisions, use of AI coding assistants, and what you would improve with additional time.
- Submit the above via this Google Form: https://forms.gle/TbcK8kXayS7UCU7h8.

Evaluation Criteria

- UI/UX and overall visual quality → 35%
- Markdown rendering accuracy → 25%
- Code quality and project organization → 20%
- Responsiveness and accessibility → 15%
- Thoughtful enhancements → 5%

We will value thoughtful prioritization and a polished core experience over implementing every possible feature.

Design Freedom

There is intentionally no reference design for this assignment.

We want to evaluate your ability to:

- Design intuitive user interfaces.
- Create a polished user experience.
- Demonstrate production-minded frontend practices within the stated time limit.

Use your own judgment for layouts, typography, colors, spacing, and interactions. Animations are optional and should only be used when they improve the experience.

Notes

- Sample markdown files are provided only for reference.
- Hidden test files will be used during evaluation.
- Your application should be resilient to unexpected or malformed input wherever reasonably possible. It does not need to repair every malformed document, but it should fail gracefully and remain usable.
- The use of AI coding assistants (e.g., ChatGPT, Claude, GitHub Copilot, Cursor, etc.) is permitted. However, you are expected to fully understand and explain your implementation during the interview.
- Work within the 4–6-hour guideline. Focus first on the required functionality and a polished core experience. Clearly identify any incomplete items or additional improvements in the README.